

Lab05_tavares

1. a

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)
```

```
# Função para leitura do arquivo em partes (chunks) filtrando o identificador da aeronave
ler_dados_aeronave <- function(tail_number, arquivo) {
  codigo_alvo <- trimws(toupper(tail_number))

  filtrar_por_tail <- function(dados_bloco, indice) {
    dados_bloco |>
      filter(trimws(toupper(TAIL_NUMBER)) == codigo_alvo)
  }

  read_csv_chunked(
    file = arquivo,
    callback = DataFrameCallback$new(filtrar_por_tail),
    chunk_size = 100000,
    col_types = cols(.default = col_character())
  )
}
```

```
)  
}
```

1. b

```
library(lubridate)  
  
# Construção da tabela tidy convertendo os tipos de dados e ordenando por data/hora  
gerar_tabela_tidy <- function(tabela_bruta) {  
  if (nrow(tabela_bruta) == 0) {  
    stop("Nenhum registro encontrado para a aeronave informada.")  
  }  
  
  tabela_bruta |>  
    mutate(  
      dep_time_num = as.numeric(DEPARTURE_TIME),  
      year_num = as.numeric(YEAR),  
      month_num = as.numeric(MONTH),  
      day_num = as.numeric(DAY)  
    ) |>  
    filter(!is.na(dep_time_num) & !is.na(year_num) & !is.na(month_num) & !is.na(day_num)) |>  
    mutate(  
      hora_partida = dep_time_num %/% 100,  
      minuto_partida = dep_time_num %% 100,  
      data_hora_partida = make_datetime(year_num, month_num, day_num, hora_partida, minuto_p  
    ) |>  
    arrange(data_hora_partida)  
}
```

1. c

```
library(leaflet)  
  
# Junção das coordenadas exatas dos aeroportos de origem e destino  
preparar_dados_geograficos <- function(tabela_ordenada, caminho_arquivo_voos) {  
  caminho_aeroportos <- file.path(dirname(caminho_arquivo_voos), "airports.csv")  
  if (!file.exists(caminho_aeroportos)) {  
    caminho_aeroportos <- "airports.csv"  
  }  
  
  tabela_aeroportos <- read_csv(caminho_aeroportos, col_types = cols(.default = col_character)  
    mutate(  

```

```

    LATITUDE = as.numeric(LATITUDE),
    LONGITUDE = as.numeric(LONGITUDE)
  )

tabela_ordenada |>
  left_join(
    tabela_aeroportos |> select(IATA_CODE, LATITUDE, LONGITUDE, AIRPORT, CITY),
    by = c("ORIGIN_AIRPORT" = "IATA_CODE")
  ) |>
  rename(lat_origem = LATITUDE, lon_origem = LONGITUDE, nome_origem = AIRPORT, cidade_origem = CITY)
  left_join(
    tabela_aeroportos |> select(IATA_CODE, LATITUDE, LONGITUDE, AIRPORT, CITY),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) |>
  rename(lat_destino = LATITUDE, lon_destino = LONGITUDE, nome_destino = AIRPORT, cidade_destino = CITY)
  filter(!is.na(lat_origem) & !is.na(lon_origem) & !is.na(lat_destino) & !is.na(lon_destino))
}

```

1. d

```

# Cálculo de estatísticas visuais e geração do mapa sequencial
gerar_mapa_trajeto <- function(tabela_geografica) {
  tabela_mapeada <- tabela_geografica |>
    mutate(
      air_time_num = as.numeric(AIR_TIME),
      distance_num = as.numeric(DISTANCE),
      velocidade_media_mph = ifelse(!is.na(air_time_num) & air_time_num > 0, (distance_num / air_time_num), NA),
      espessura_linha = ifelse(
        is.na(velocidade_media_mph), 2,
        2 + 5 * (velocidade_media_mph - min(velocidade_media_mph, na.rm = TRUE)) /
          (max(velocidade_media_mph, na.rm = TRUE) - min(velocidade_media_mph, na.rm = TRUE))
      )
    )

  aeroportos_unicos <- bind_rows(
    tabela_mapeada |> select(codigo = ORIGIN_AIRPORT, nome = nome_origem, cidade = cidade_origem),
    tabela_mapeada |> select(codigo = DESTINATION_AIRPORT, nome = nome_destino, cidade = cidade_destino)
  ) |> distinct(codigo, .keep_all = TRUE)

  mapa_base <- leaflet() |>
    addTiles() |>
    addCircleMarkers(

```

```

    data = aeroportos_unicos,
    lng = ~lon, lat = ~lat,
    popup = ~paste0("<b>Aeroporto: </b>", codigo, " - ", nome, "<br><b>Cidade: </b>", cidade),
    radius = 4, color = "#2B8CBE", fillOpacity = 0.8
  )
}

for (i in seq_len(nrow(tabela_mapeada))) {
  voo_atual <- tabela_mapeada[i, ]
  mapa_base <- mapa_base |>
    addPolylines(
      lng = c(voo_atual$lon_origem, voo_atual$lon_destino),
      lat = c(voo_atual$lat_origem, voo_atual$lat_destino),
      weight = voo_atual$espessura_linha,
      color = "#E31A1C",
      opacity = 0.5,
      popup = paste0("<b>Etapa Cronológica: </b>#", i, "<br>",
                     "<b>Data/Hora Partida: </b>", voo_atual$data_hora_partida, "<br>",
                     "<b>Rota: </b>", voo_atual$ORIGIN_AIRPORT, " -> ", voo_atual$DESTINATION_AIRPORT, "<br>",
                     "<b>Velocidade Média: </b>", ifelse(is.na(voo_atual$velocidade_media), "N/A", voo_atual$velocidade_media), "</b>")
    )
}

return(mapa_base)
}

# Função principal solicitada
analisa_aeronave <- function(tail_number, arquivo) {
  tabela_bruta <- ler_dados_aeronave(tail_number, arquivo)
  tabela_tidy <- gerar_tabela_tidy(tabela_bruta)
  tabela_geo <- preparar_dados_geograficos(tabela_tidy, arquivo)
  mapa_final <- gerar_mapa_trajeto(tabela_geo)

  return(list(
    tabela = tabela_tidy,
    mapa = mapa_final
  ))
}

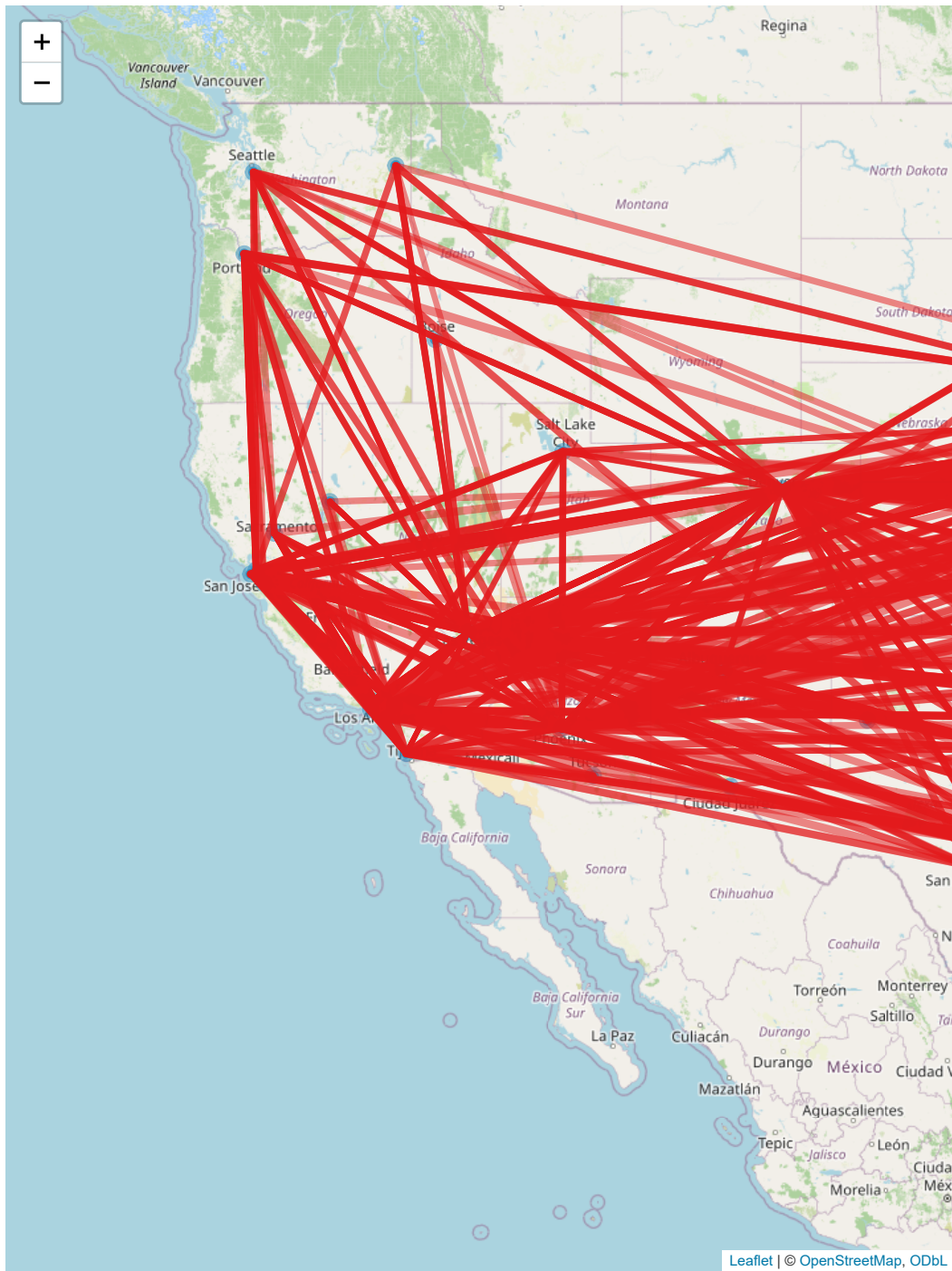
# Execução da função
resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = "flights.csv.zip"
)

```

```
)
```

```
# Renderiza o mapa diretamente ao executar a Chunk 1d  
resultado$mapa
```

```
Warning in file.info(file): cannot resolve owner of file  
'\\smb\ra237053\Documentos\file2efc3f4c3634\webshot2efc26b07966.pdf': Não foi  
feito mapeamento entre os nomes de conta e as identificações de segurança
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 16:55:35 -03"
```